

Markowitz Portfolio Optimization
Project Report

Pierre Cotraud
December 2025

Contents

1 Simple least-squares problem 1
2 Markowitz portfolio selection 1
2.1 Classical problem 1
2.2 Probabilistic variant 2
2.3 Effect of extending the test period to 2021 3
2.4 Time-series of asset returns 3
2.5 Monte Carlo analysis 4
3 Resolving the Kurtosis Problem 5
3.1 A Simple Nonconvex Test Problem 5
3.2 Solving the Kurtosis Problem with CVXPY (SCP) 6
3.3 Solving the Kurtosis Problem with SLSQP 6
3.4 Monte Carlo Comparison between Markowitz and Kurtosis Methods 7
3.5 Convex Relaxation 8
4 Conclusion 8

1 Simple least-squares problem

We start by solving the classical **convex least-squares optimization problem**:

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2,$$

where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Comparison of computational speeds

We implemented numerical solvers using two different optimization frameworks: **CVXPY** and **SciPy**. To compare both methods, we generated random instances of increasing sizes (m, n) and measured the computational time required to solve each instance. Figure 1 summarizes the results.

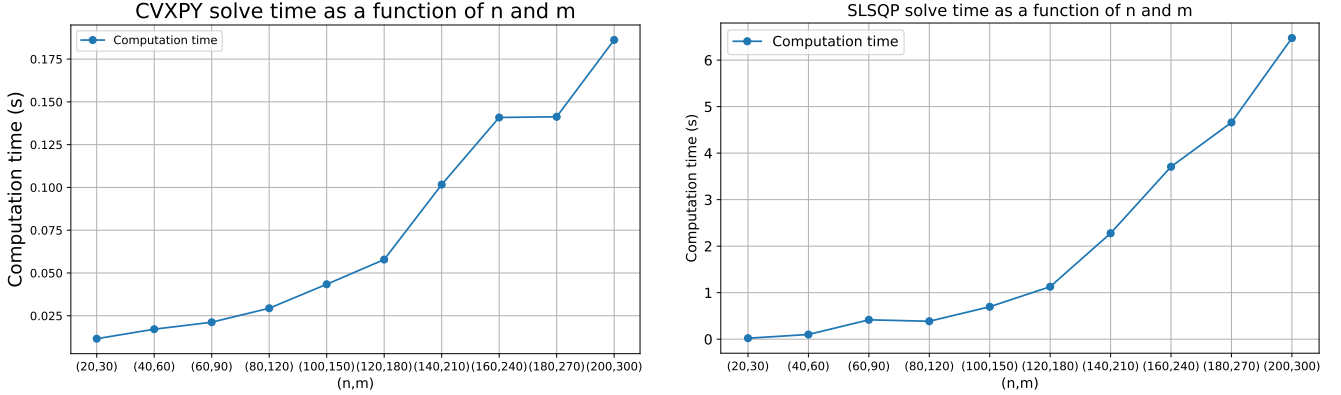


Figure 1: Comparison of computational times for CVXPY and SciPy on least-squares problems.

For this test, SLSQP is faster for small (n, m) , while CVXPY's runtime grows more slowly and overtakes SLSQP for larger (n, m) .

2 Markowitz portfolio selection

2.1 Classical problem

We now consider the classical Markowitz mean–variance portfolio optimization problem:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & x^\top \Sigma x \\ \text{s.t.} \quad & \mu^\top x \geq r_{\min}, \\ & \mathbf{1}^\top x = 1, \\ & x \geq 0. \end{aligned} \tag{1}$$

We set the minimum acceptable return to:

$$r_{\min} = \frac{1.6}{252}.$$

We implemented a solver `markovitz_portfolio(mu, sigma)` that returns the optimal x^* . We evaluate performance using:

$$(s_{\text{tr}}, s_{\text{te}}) = \text{compute_metrics}(x^*, Y_{\text{train}}, Y_{\text{test}}).$$

We begin with a 3-asset universe and then extend to 20 assets to analyse diversification effects.

Table 1: Metrics for $n_{\text{assets}} = 3$

Training (2016–2019)	
Return	19.724704
Std. Dev.	16.493868
Variance	1.079554
Testing (2020)	
Return	0.360041
Std. Dev.	37.211168
Variance	5.494726

Table 2: Metrics for $n_{\text{assets}} = 20$

Training (2016–2019)	
Return	12.902767
Std. Dev.	10.434147
Variance	0.432029
Testing (2020)	
Return	5.404807
Std. Dev.	32.349382
Variance	4.152708

Case $n_{\text{assets}} = 3$. The portfolio exhibits very strong training performance (high mean return, moderate variance), but collapses in the testing: the mean return is close to zero and the variance explodes.

Case $n_{\text{assets}} = 20$. Increasing the number of assets reduces this effect: the training return decreases but the variance drops substantially. The testing performance improves from around 0.36% to 5.40%. This illustrates the diversification benefit of enlarging the investment universe.

Remark. Out-of-sample instability is amplified by COVID-induced volatility. Markowitz theory implicitly assumes a form of statistical financial stability, which was violated in 2020.

2.2 Probabilistic variant

We now incorporate a probabilistic loss constraint (which we transform under Gaussian assumptions):

$$\Pr(r \leq \alpha) \leq \beta,$$

with initial choices $\beta = 0.49$ and $\alpha = \frac{1.6}{252}$.

Table 3: Probabilistic Markowitz performance for 3 and 20 assets

# Assets	Set	Mean Return (%)	Std. Dev. (%)	Variance
3	Train	19.72	16.49	1.08
3	Test	0.36	37.21	5.49
20	Train	12.90	10.43	0.43
20	Test	5.40	32.35	4.15

Using the probabilistic solver `markovitz_portfolio_probabilistic` with 3 assets and 20 assets, we obtain values that are very close to those of the standard Markowitz model.

We can now discuss the effect of the parameter β on our model. As β decreases, the probabilistic constraint becomes increasingly restrictive. For values below 0.47, the optimiser is no longer able to find a feasible solution: the risk constraint is simply too stringent to be satisfied by any admissible portfolio.

Conversely, when β increases, the constraint becomes looser, and the optimisation is allowed to select portfolios that are less restricted by downside risk (although our solver cannot handle values of $\beta \geq 0.5$). Larger values of β therefore lead to more aggressive allocations, whereas smaller values enforce more conservative portfolios.

2.3 Effect of extending the test period to 2021

In the previous questions, the test period was restricted to the year 2020, which was strongly affected by the COVID-19 crisis and led to unusually high volatility. We now extend the test period to include 2021 and recompute the metrics for the probabilistic Markowitz portfolio (20 assets, $\beta = 0.49$, $\alpha = 1.6/252$).

Table 4: Probabilistic Markowitz (20 assets): test on 2020 vs 2021

Year	Set	Mean Return (%)	Std. Dev. (%)	Variance
2016-2019	Train	12.90	10.43	0.43
2020	Test	5.40	32.35	4.15
2021	Test	27.54	11.00	0.48

When testing only on 2020, the portfolio exhibits a moderate positive mean return ($\approx 5.4\%$) but with very high volatility (variance ≈ 4.15), reflecting the extreme market conditions during the COVID shock. When the test period is moved to 2021, the mean test return increase to about 27% while the variance drops to about 0.48, indicating a much more favourable and stable environment. This confirms that the poor 2020 performance is largely driven by an exceptional crisis year rather than by an intrinsic failure of the portfolio construction.

2.4 Time-series of asset returns

In order to better understand the data used in our Markowitz and probabilistic optimizations, we plot the time-series of the daily returns for 3 assets.

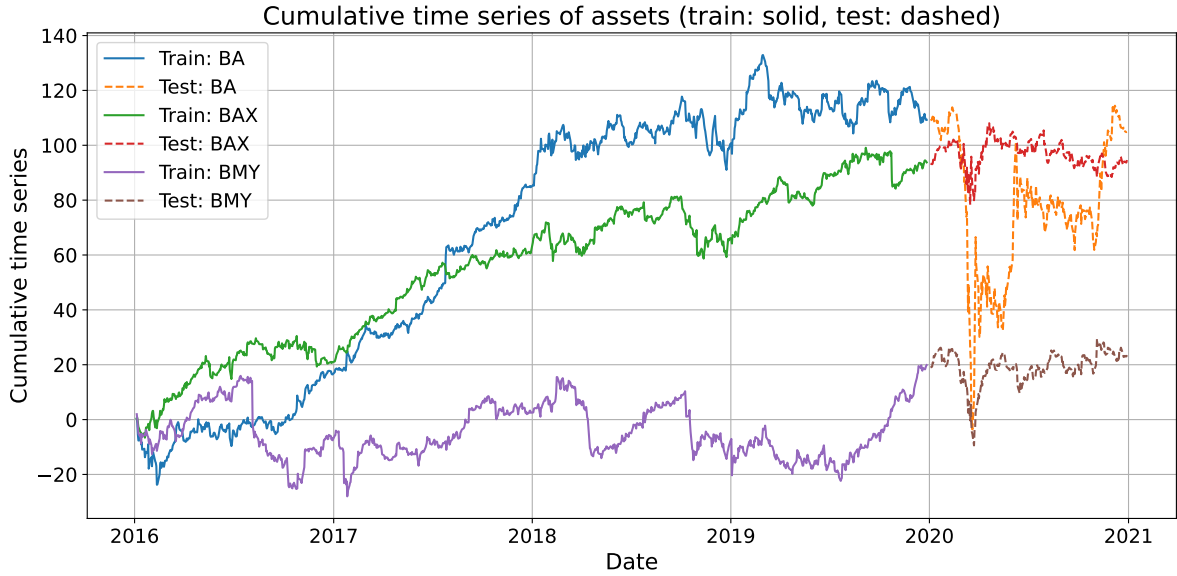


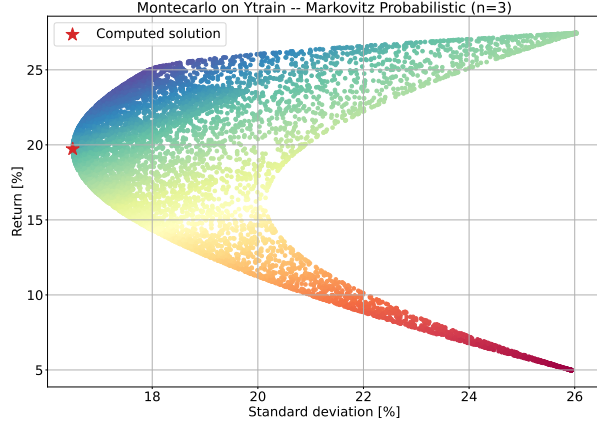
Figure 2: Daily return time-series of 3 assets in the training (2016–2019) and testing (2020) periods.

These plots visually confirm the statistical observations made earlier: 2020 is an outlier year with much higher risk, which helps to explain why some portfolios that perform well in-sample (on 2016–2019) can behave poorly out-of-sample when tested on 2020.

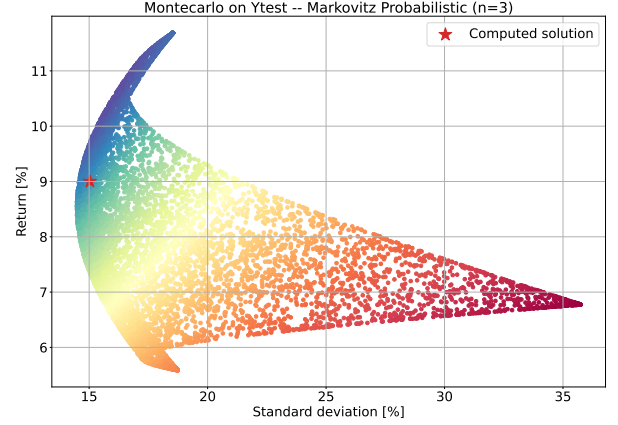
For this reason, from now on simulations will be conducted using data from the year 2021. This will give us a better view on the portfolio’s performance, with a “normal” year. We did so because these theories are based on a certain level of economic stability, and we wanted to remain consistent with that assumption.

2.5 Monte Carlo analysis

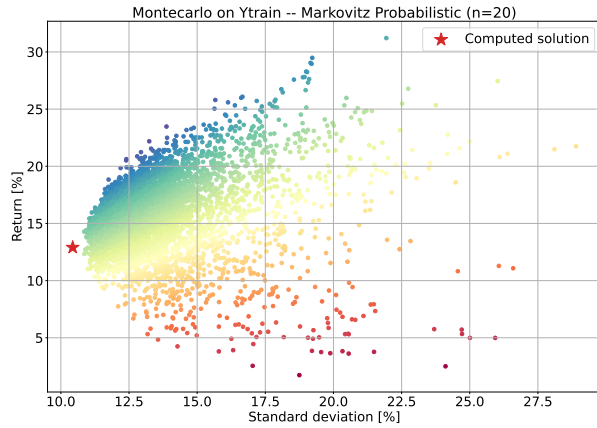
To further assess the robustness of the Markowitz solution, we perform a Monte Carlo analysis: for each case, we simulate many possible realizations of asset returns and plot the resulting portfolios in the risk–return plane (standard deviation vs. mean return), together with the previously computed portfolio.



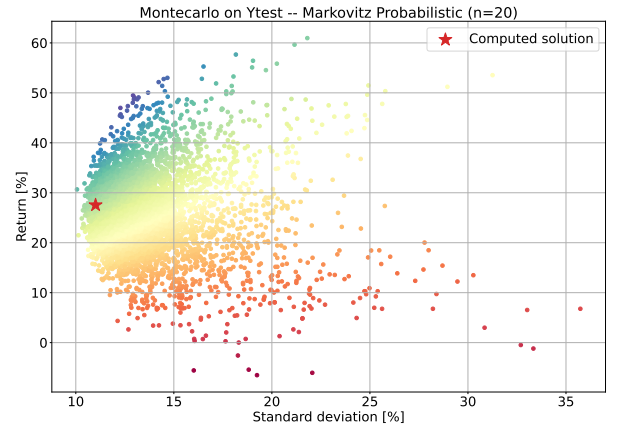
(a) Training, 3 assets (2016–2019)



(b) Testing, 3 assets (2021)



(c) Training, 20 assets (2016–2019)



(d) Testing, 20 assets (2021)

Figure 3: Monte Carlo simulations in the standard deviation vs. return plane for 3 and 20 assets (training and testing sets). The red point marks the computed Markowitz solution.

For the case with three assets (Figures 3a–3b), the training solution lies on the efficient frontier. The return–risk ratio is maximized. On the testing dataset, the solution is nearly optimal. The optimal point should lie in the upper-left region of the cloud, where portfolios with higher return and lower risk are observed.

For the case with twenty assets (Figures 3c–3d), the Monte Carlo cloud becomes more concentrated, and the Markowitz solution remains close to the efficient frontier of the simulated portfolios. This indicates improved diversification: for a comparable risk level, the sampled portfolios cluster more tightly around the chosen solution. Out-of-sample variability is reduced, confirming the stabilizing effect of increasing the number of assets.

3 Resolving the Kurtosis Problem

The kurtosis minimization problem can be formulated as:

$$\begin{aligned}
& \min_{x \in \mathbb{R}^n, X \in \mathbb{S}^n, z \in \mathbb{R}^{n(n+1)/2}, g \in \mathbb{R}} g \\
& \text{s.t.} \quad X = xx^\top, \quad z = L_2 \text{vec}(X), \quad \left\| (S_2 \Sigma_4 S_2^\top)^{1/2} z \right\|_2 \leq g, \\
& \quad \sum_{i=1}^n x_i = 1, \quad x \geq 0, \quad \mu^\top x \geq r_{\min}.
\end{aligned} \tag{2}$$

All constraints in Problem (2) are affine or convex *except* the relation

$$X = xx^\top.$$

To see this, consider the case $n = 2$ and define

$$x_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad X_1 = x_1 x_1^\top = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad x_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad X_2 = x_2 x_2^\top = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

Both (x_1, X_1) and (x_2, X_2) satisfy $X = xx^\top$. Consider now their midpoint:

$$\bar{x} = \frac{1}{2}(x_1 + x_2) = \begin{pmatrix} 1/2 \\ 1/2 \end{pmatrix}, \quad \bar{X} = \frac{1}{2}(X_1 + X_2) = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \end{pmatrix}.$$

We have

$$\bar{x} \bar{x}^\top = \begin{pmatrix} 1/2 \\ 1/2 \end{pmatrix} \begin{pmatrix} 1/2 & 1/2 \end{pmatrix} = \begin{pmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{pmatrix} \neq \bar{X},$$

Therefore, there is a point between (x_1, X_1) and (x_2, X_2) that does not satisfy $X = xx^\top$.

To address the kurtosis minimisation problem, we used Sequential Quadratic Programming (SQP) and compared two implementations: a custom SQP scheme implemented through CVXPY, and the SLSQP algorithm from SciPy. To do so, we started with a simple, controlled nonconvex optimisation problem.

3.1 A Simple Nonconvex Test Problem

We first considered the following toy example:

$$\begin{aligned}
& \min_{x \in \mathbb{R}^n} \quad \frac{1}{2} \|Ax - b\|_2^2 \\
& \text{s.t.} \quad 0 \leq x \leq 1, \\
& \quad \|x\|_2^2 \geq \frac{1}{2}.
\end{aligned} \tag{3}$$

This problem is nonconvex because of the quadratic constraint. The goal here was not the model itself but a benchmark: to compare SQP (CVXPY) and SLSQP (SciPy) on a simple nonconvex instance.

On this simple test problem, both solvers reach essentially the same objective value. SLSQP is faster but requires more iterations, whereas CVXPY-SQP converges in fewer steps with a higher per-iteration cost. In both cases, constraints are satisfied up to numerical precision.

Figure 4 shows that the SQP method successfully converges to a constraint-satisfying solution, which is encouraging before tackling the kurtosis problem.

Table 5: Comparison Between SQP (CVXPY) and SLSQP (SciPy) on the Nonconvex Problem (3)

Solver	Time (s)	Iterations	Optimum Value	Constraint Violation
SQP (CVXPY)	0.054	7	9.918843	9.99×10^{-15}
SLSQP (SciPy)	0.013	17	9.918843	3.48×10^{-8}

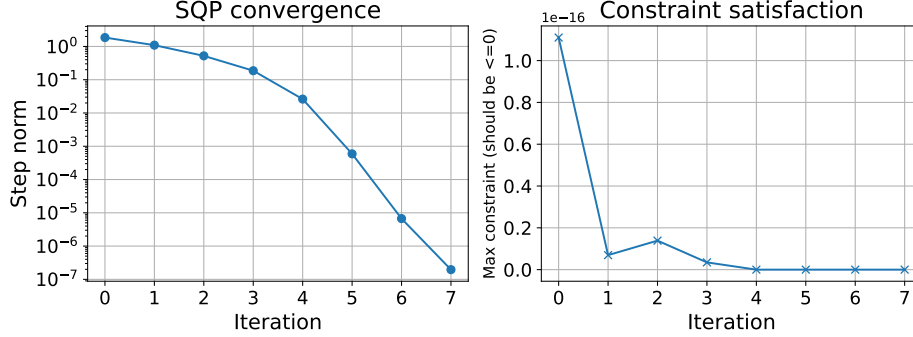


Figure 4: Norm at each step and constraint for the CVXPY method.

3.2 Solving the Kurtosis Problem with CVXPY (SCP)

To handle this nonconvexity, we attempted a Sequential Convex Programming (SCP) approach: at each iteration, the nonconvex constraint is linearised around the current point, yielding a convex subproblem solved with CVXPY.

The solution does converge, but it requires a large number of iterations. For example, with just 3 assets and 200 iterations, the algorithm reaches an optimum of 2.71, whereas the true optimum is 2.61.

We can also confirm that the solution does not depend on the initial condition, as shown in Figure 5. The portfolio obtained has also been compared to the Markowitz one: the optimal x is not the same.

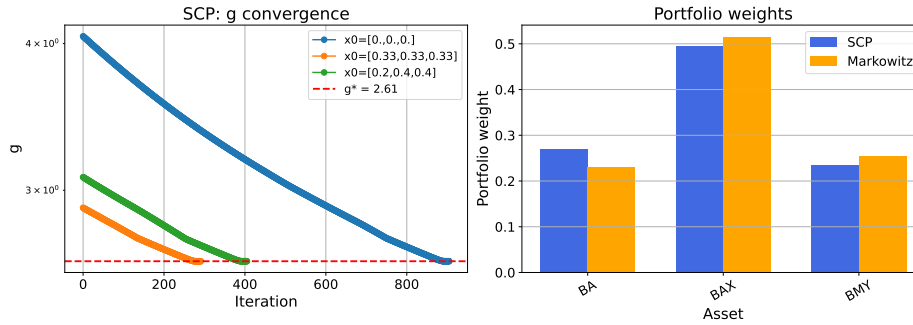


Figure 5: Convergence of g for different initial conditions and comparison of portfolios.

Method	Time (s)	g	Optimal Portfolio x
SCP	116.112	2.67294	[0.3436, 0.4191, 0.2373]

Table 6: SCP optimisation results for 200 iterations

3.3 Solving the Kurtosis Problem with SLSQP

We then applied the SLSQP solver from SciPy to Problem (2). All variables were packed into a single decision vector, from which the portfolio weights x were extracted inside the objective and constraints.

Method	Time (s)	g	Optimal Portfolio x
SLSQP	0.1298	2.61005	[0.2704, 0.4943, 0.2353]

Table 7: SLSQP optimisation results on the kurtosis problem for 3 assets

SLSQP converged reliably and provided a feasible optimal solution within a short computing time.

The SCP approach is significantly slower than SLSQP. In practice, it would require at least 1,000 iterations to obtain a solution comparable to SLSQP, which is not a realistic option.

3.4 Monte Carlo Comparison between Markowitz and Kurtosis Methods

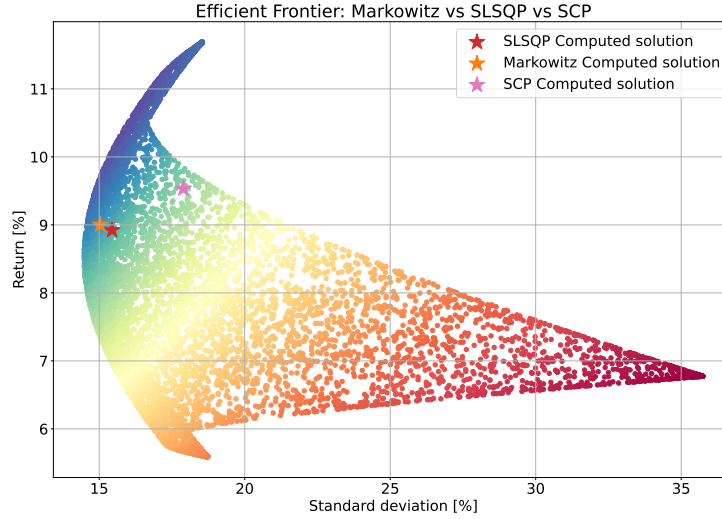


Figure 6: Monte Carlo simulations for 3 assets with the Markowitz, SCP and SLSQP methods (2021 on Y_{test}).

In Figure 6, we plotted the optimal points obtained from all the methods. We restricted the test to 3 assets; otherwise, the SCP method would require excessive computation time.

By comparing the solutions, we observe that the Markowitz optimum actually dominates both SLSQP and SCP in terms of the risk–return trade-off. In the scatter plot, the Markowitz point lies further toward the upper-left region, meaning that it achieves a better return for the same level of risk, or lower risk for the same return.

The solution obtained by SLSQP does not improve upon Markowitz and even appears slightly less efficient, which is expected since SLSQP attempts to solve the nonconvex kurtosis-based problem and can get trapped in a suboptimal region.

Finally, the SCP solution is visibly far from optimal — but this is mainly due to the limited number of iterations allowed (200 in this experiment), which prevents SCP from converging properly to a competitive solution.

3.5 Convex Relaxation

We consider the following problem:

$$\begin{aligned}
& \min_{x \in \mathbb{R}^n, X \in \mathbb{S}^n, z \in \mathbb{R}^{n(n+1)/2}, g \in \mathbb{R}} g \\
& \text{s.t.} \quad \sum_{i=1}^n x_i = 1, \quad x \geq 0, \quad \mu^\top x \geq r_{\min}, \\
& \quad \begin{bmatrix} X & x \\ x^\top & 1 \end{bmatrix} \succeq 0, \quad z = L_2 \text{vec}(X), \quad \left\| (S_2 \Sigma_4 S_2^\top)^{1/2} z \right\|_2 \leq g.
\end{aligned} \tag{4}$$

Let $\mathcal{P}_2$ and $\mathcal{P}_4$ denote the feasible sets of Problems (2) and (4), respectively. If (x, X) is feasible for (2), then $X = xx^\top$ implies

$$\begin{bmatrix} X & x \\ x^\top & 1 \end{bmatrix} = \begin{bmatrix} x \\ 1 \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix}^\top \succeq 0,$$

so (x, X) is also feasible for (4). Hence $\mathcal{P}_2 \subseteq \mathcal{P}_4$. Since both problems minimize the same objective g over these sets, we have

$$\min_{(x, X, \dots) \in \mathcal{P}_4} g \leq \min_{(x, X, \dots) \in \mathcal{P}_2} g.$$

For this problem, the convex relaxation yields

$$x^* = [0.27044373, 0.49425587, 0.23530040]^\top, \quad g^* = 2.6100472879670127.$$

with a computational time of approximately 0.0283 seconds (about ~ 10 times faster). These values are essentially identical to those obtained with SLSQP, both in terms of the optimal portfolio weights and the objective value, so the quality of the solution on the *training* data is the same as for SLSQP. Consequently, the testing behaviour is also unchanged.

We also check the consistency of the lifting constraint $X = xx^\top$. Numerically, we obtain

$$\|X - x(x)^\top\| = 4.70 \times 10^{-9},$$

which is at the level of machine precision. This shows that, at the optimum, the convex relaxation is *tight*, so solving the relaxed Problem (4) effectively recovers a solution of the original nonconvex Problem (2).

4 Conclusion

In this project, we compared several approaches to portfolio optimisation, ranging from classical least squares and Markowitz mean–variance models to probabilistic constraints and kurtosis-based formulations. Our experiments confirmed well-established effects: diversifying across more assets systematically improves out-of-sample performance, and the results are highly sensitive to the choice of training and testing windows—particularly during crisis periods such as 2020.

The experiments also showed that incorporating higher-moment information (kurtosis) is beneficial in terms of the return–risk trade-off. We tested three methods: SCP, SLSQP, and the relaxed formulations. SCP demonstrated clear limitations in terms of computational time, requiring far more iterations to converge. In contrast, SLSQP and the relaxation approach proved highly efficient for this problem, and the optima they produced appear consistent and reliable for our setting.

Machine learning could in principle improve the return/variance trade-off by providing better estimates of expected returns, covariances, or downside-risk measures (for example through non-linear factor models or regime-switching predictors). In practice, however, this is extremely challenging: financial signals have a very low signal-to-noise ratio, structural breaks are frequent, and the available data are limited compared to the number of parameters in flexible models. As a result, complex ML models tend to overfit past data, making it very hard to obtain robust and genuinely superior portfolios in real markets.